

Instructor: Date:

Learner: Regional Branch:

Program: Center:

CONTENT

- Let's Start! • Let's Explore! • Let's Practice! • Extension Activity • Target Words • Learning Points



Let's Start!

Data Structures



Learning Outcome:

Describe common activities at work to a visitor using short sentences in familiar spoken and written situations.
Understand information about data and its structure in simple spoken and written situations.

Ask your partner:

- What problems can you relate to data structures?
- What are some issues related to the implementation of data structures?

Can Do Statements:

- I can talk about the implementation issues related to data structures.
- I can explain the possible implementation issues presented in data structures.
- I can describe certain issues related to data structures.





Let's Explore!

Understanding Data Structures



Instructions: Read the text below.

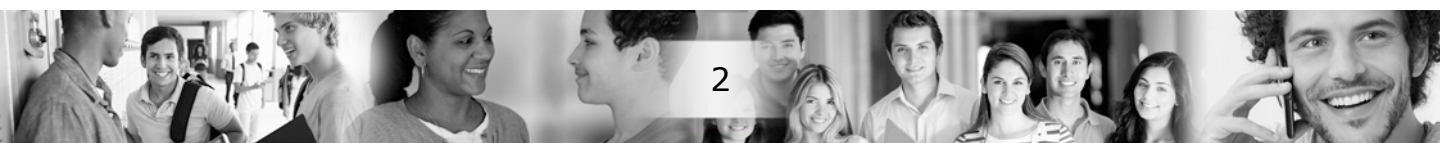
Today, I'm talking about the implementation issues related to data structures. First, I'm explaining them. They are a way of arranging data to be used in a computer program. Data structures use some basic components: these are basic data types, which are ways to organize these data items among them; and a set of operations to manipulate the data. Some of the common structures implemented in computer science are arrays and lists. As you may know, the main difference is that an array has no internal links among its elements as the lists do.

Let's talk about the arrays. They are data structures that consist of basic data types, although all data must be of the same type. The data distribution has to be arranged by storing it in sequent locations.

Now, talking about the implementation issues presented in data structures, we can say that any implementation of data structures is based on the language syntax of the programming as well as the type of data and available features. Also, the algorithms needed for data operations which manipulate the structures are vital. Some implementation issues are defined with procedural languages as C or object-oriented languages as C++.

In procedural languages as C, the data structure is placed in one part of the program, and the operations are confined separately. There is no way to link them. On the other hand, object-oriented languages as C++ have a data structure and function declarations in the same entity called a class. This means that the implementation and use of the data structure is controlled by the designer.

Data structures and algorithms create the most important part of computer science. Different variations of fundamental data structures exist if we understand them and know how to use them. Then, it is easy to make sure that program design has good quality and avoids failures.



**Let's Practice!**

Data structures are the most important part of computer science.

Instructions: Read the paragraph. Underline all the mistakes in the paragraph.

Each programming implementations has its data structure. The data structure depends on the programming language syntax, the data types, and the algorithms for the data operations. Algorithms and data structures is the heart of computer science. The algorithms that we need to manipulate the structure is vital.

**Let's Practice!**

Algorithms and Data Structures

Instructions: Organize the mystery sentence.

1. are / data / on / variations / fundamental / structures. / many / There / the

2. them. / use / specific / programmers / to / syntax / a / The / write

3. the / algorithm / to / manipulate / They / too. / them / choose



Extension Activity

What data structure is more common?

Instructions: Complete the following steps. Fill in the Self-Assessment.

- Research about extra implementation issues in different programming languages.
- Write your findings about implementation issues in different programming languages.
- Share your findings with your teacher and your classmates.

Self-Assessment

Yes

No

- I research about implementation issues in different programming languages.
- I write about implementation issues in different programming languages.
- I talk about implementation issues in different programming languages.

☐ ☐
☐ ☐
☐ ☐


Target Words

- **issues** (noun): the distribution of a group or series
e.g. I'm talking about the implementation issues related to data structures.
- **arrange** (verb): to organize
e.g. They are a way of arranging data to be used in a computer program.
- **failure** (noun): something with lack of success
e.g. Different variations of fundamental data structures exist, if we understand them and know how to use them, then is easy to make sure that program design has good quality and avoids failures.
- **link** (noun): something that connects one thing to another
e.g. As you may know, the main difference is that an array has no internal links among its elements as the lists do.
- **sequent** (adjective): consecutive
e.g. The data distribution has to be arranged by storing it in sequent locations.
- **confine** (verb): to put in limits
e.g. In procedural languages as C, the data structure is placed in one part of the program, and the operations are confined separately.



Learning Points

Nouns That Can Be Countable and Uncountable

There are some nouns that can be used as **countable** or **uncountable**.

Sometimes a noun can be used as a **countable** or **uncountable** noun; its meaning may change depending on its usage.

Noun	Countable	Uncountable
Aid	Something that helps you to perform something	Help given to people in need
Business	Companies	The activity of selling and buying goods
Danger	Something that can harm	The possibility of something bad occurring
Distribution	Geographic range of language, disease, etc.	Arrange of things as information
Failure	Something that is not successful	Lack of success
Quality	A characteristic of something or someone	How good something is

Examples

- *The data distribution has to be arranged by storing it in sequent locations.*
- *Different variations of fundamental data structures exist if we understand them and know how to use them. Then, it is easy to make sure that program design has good quality and avoids failures.*



Learning Points

Simple Present and Present Continuous with Direct and Indirect Objects in Affirmative

You can relate **indirect objects** to **direct objects** in a sentence, as a way to avoid repetition.

Simple Present

To create an affirmative sentence in **simple present** using **direct objects**, you need to use the following structure.

Direct objects always follow action verbs that need something or someone to receive the action.

Pronoun + Verb + **Complement (direct object)**

Example:

*I want to know more about **data structures**.*

To create an affirmative sentence in **simple present** using indirect objects, you need to follow the next structure.

Indirect objects typically refer to the **direct object**.

Pronoun + Verb + **Indirect object** + Complement

Example:

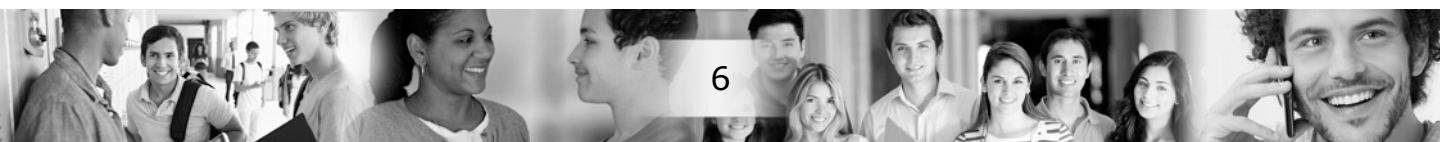
*I want to know more about **data structures** and how we relate **them** to different fields in software programming.*

Present Continuous

To create an affirmative sentence in **present continuous** using **direct objects**, you need to follow the next structure.

Direct objects always follow action verbs that need something or someone to receive the action.

Pronoun + Verb to be + Verb (ing) + **Complement (direct object)**





Learning Points

Example:

*I am checking all the **issues** with data implementation.*

To create an affirmative sentence in **present continuous** using **indirect objects**, you need to follow the next structure.

Indirect objects typically refer to the direct object.

Pronoun + Verb to be + Verb (ing) + **Indirect object** + Complement

Example:

*I am checking all the **issues** with data implementation because we are finding solutions to **them** in the next meeting.*

Examples

- *Today, I'm talking about the implementation issues related to data structures. First, I'm explaining them.*
- *Different variations of fundamental data structures exist if we understand them and know how to use them; then, it is easy to make sure that program design has good quality and avoids failures.*